

UNIVERSITY OF
GEORGIA
College of Engineering
School of Electrical and Computer Engineering

ECSE 2920: Design Methodology

Tronics Industries: Electronic Test Bench (E.T.B)

Technical Documentation

Avery Dedjoe
Allan Gramajo
Justin Park
Samuel Baker
Jakob Braski

April 22, 2026

Table of Contents

1. Objectives.....5
1.1 Project Objectives.....5
1.2 Functional Objectives.....5
1.2.1 Core Platform Requirements.....5
1.2.2 Instrumentation Capabilities.....5
A. Function Generator.....6
C. Voltmeter.....6
D. DC Voltage Reference.....6
E. Frequency Measurement (A- Level Requirement).....6
1.3 User Interface Requirements.....6
2. Constraints.....7
2.1 Electrical Constraints.....7
2.2 Power Constraints.....8
2.3 Manufacturability Constraints.....8
3. Hardware Design.....9
3.1 Power Supply Design.....9
3.1.1 Design Objective.....9
3.1.2 Topology Selection.....9
3.1.3 Virtual Ground Generation.....9
3.1.4 Crossover Distortion Mitigation.....9
3.1.5 Stability and Compensation.....10
3.2 6-Bit SAR ADC Implementation.....12
3.2.1 Architecture Selection.....12
3.2.2 R–2R Ladder Network.....12
3.2.3 Comparator Selection.....12
3.2.4 SAR Algorithm.....13
3.3 Digital Potentiometer Subsystem.....15
3.3.1 Device Selection.....15
3.3.2 Calibration Methodology.....15
3.4 Rotary Encoder Design.....17
3.4.1 Rotary Encoder Principles.....17
3.4.2 Integration.....17

3.5 Ohmmeter Subsystem.....19
3.5.1 Design Objective and Theory of Operation.....19
3.5.2 Component Selection and Sensitivity Optimization.....19
3.5.3 Hardware Configuration: Rheostat Mode.....19
3.5.4 Resolution and Error Analysis.....20
3.6 DC Voltage Reference Subsystem.....21
3.6.1 Design Objective and Scaling Logic.....21
3.6.2 Difference Amplifier Topology.....21
3.6.3 Signal Conditioning and Stability.....22
3.6.4 Hardware Components and Decoupling.....22
3.7 Voltmeter Subsystem.....24
3.7.1 Successive Approximation (SAR).....24
3.7.2 Hardware Input Protection & Scaling.....24
3.7.3 Hardware-Level Accuracy & Noise Reduction.....24
3.8 Function Generator Subsystem.....26
3.8.1 Square Wave Generation Path.....26
3.8.2 Sine Wave Generation Path (PWM-to-Analog).....26
3.8.3 Final Gain and DC Offset Stage.....26
3.8.4 Hardware-Level Calibration.....27
3.9 Frequency Measurement Subsystem.....28
3.9.1 The Schmitt Trigger.....28
3.9.2 Logic Level Protection & Scaling.....28
3.9.3 Pulse Detection Strategy.....28
3.9.4 Hardware Performance Specs.....28
4. Software Design.....30
4.1 Power Subsystem Software & Initialization.....30
4.1.1 Boot Sequencing & Stabilization.....30
4.1.2 Virtual Ground Offset Compensation.....30
4.1.3 Power Management & Thermal Safety.....30
4.2 User Interface & State Machine Logic.....31
4.2.1 Interrupt-Driven Input Handling.....31
4.2.2 State Machine Architecture.....31
4.2.3 Hierarchical State Machine (FSM) Architecture.....31
4.2.4 LCD Refresh Optimization.....32
4.3 Measurement Algorithms.....33
4.3.1 Binary Search Logic (Ohmmeter).....33
4.3.2 Successive Approximation (Voltmeter).....34
4.3.3 Data Averaging & Smoothing.....34
4.3.4 Resistance Calibration Mapping.....35

4.4 Signal Generation Logic....36
4.4.1 Square Wave Generation (Hardware PWM)....36
4.4.2 Sine Wave Generation (Sinusoidal PWM)....36
4.4.3 Frequency-Dependent Amplitude Compensation....36
4.4.4 Signal Cleanup & Safety....37
4.5 Frequency Measurement Logic....39
4.5.1 Edge Detection and Interrupt Handling....39
4.5.2 Mathematical Conversion and Period Calculation....39
4.5.3 Signal Stability Logic....39
4.5.4 UI Threading and Refresh Rate....40
5 PCB Design and Layout Strategy....41
5.1 Layout Strategy & Signal Integrity....41
5.2 Thermal Management....41
5.3 Design for Manufacturability (DFM)....41
5.4 CAD Design Rules Check (DRC)....42
6. Ethics....44
7. Self-Critique....45
7.1 Revision Plan....46
7.2. Individual Reflections....47
8. Appendix....48

1. Objectives

1.1 Project Objectives

The objective of the ECSE 2920 Electronics Tool Bench project is to design, implement, and validate a multi-function electronic testbench platform controlled entirely by the Raspberry Pi 4 (RP4).

The system is intended to replicate the essential functionality of laboratory-grade instrumentation, including oscilloscopes, function generators, power supplies, ohmmeters, and voltmeters, within a compact, low-cost, microcontroller-based architecture.

The RP4 serves as the sole processing unit responsible for:

- Signal generation
- Data acquisition
- User interface control
- SPI/I2C communication
- System state management

No additional microcontrollers or processors are permitted.

1.2 Functional Objectives

The Electronics Tool Bench must achieve the following high-level capabilities:

1.2.1 Core Platform Requirements

1. Verify and utilize the Pi-Protection-Hat (PPH) for safe GPIO interfacing.
2. Characterize and program dual digital potentiometers using SPI.
3. Implement a rotary encoder-driven user interface using an LCD display.
4. Develop structured Python-based control software.
5. Derive all necessary system voltages from a provided 24 V supply.

1.2.2 Instrumentation Capabilities

The completed system must implement the following instrumentation features:

A. Function Generator

- Square wave generation:
- Sine wave generation (A- level requirement):

B. Ohmmeter

C. Voltmeter

D. DC Voltage Reference

E. Frequency Measurement (A- Level Requirement)

1.3 User Interface Requirements

The interface must:

- Use rotary encoder rotation for navigation.
- Use encoder push-button for selection.
- Implement “Back” and “Main” options at each level.
- Automatically disable outputs when leaving active output screens.
- Display live measurement or output data where required.

All UI behavior must be implemented exclusively through the rotary encoder. LCD onboard buttons are not permitted.

2. Constraints

2.1 Electrical Constraints

The design was bounded by the electrical limitations of the Raspberry Pi 4:

- 3.3 V GPIO logic tolerance (not 5 V tolerant)
- Limited current sourcing capability
- SPI timing and clock constraints
- Ground reference sensitivity

These constraints directly influenced:

- Comparator pull-up design
- Level shifting considerations
- Power stage isolation
- R–2R ladder voltage reference scaling

The user interface must follow a strict hierarchical tree structure. Deviations are not permitted.

I. OFF: Everything is off
II. Mode Select
    A. Function Generator
        1. Type
            a) Sine
            b) Square
            c) Back
            d) Main
        2. Frequency
            a) Input Frequency
            b) Back
            c) Main
        3. Amplitude
            a) Input Amplitude
            b) Back
            c) Main
        4. Output
            a) On (note if you back out of this or go to main, must turn off by default).
            b) Off
            c) Back
            d) Main
    B. Ohmmeter (UI shows value of reading)
        1. Back
        2. Main
    C. Voltmeter (UI show value of reading)

1. Source
    a) External
    b) Internal Reference (If selected, go to DC Reference)
    c) Back
    d) Main

D. DC Reference
    1. Voltage Value Input
    2. Output
        a) On (note if you back out of this or go to main, must turn off by default).
        b) Off
        c) Back
        d) Main
    3. Back
    4. Main

E. Frequency Measurement
    1. Back
    2. Main

F. Back

G. Main

2.2 Power Constraints

- Input supply fixed at 24 V DC
- Virtual ground target of ±12 V
- Op-amp output swing limitations
- Stability under varying load conditions

The 24 V constraint necessitated a virtual ground topology rather than dual supply generation. The project required a minimum 5-bit ADC resolution. An 6-bit architecture was selected to:

- Improve quantization granularity
- Demonstrate deeper understanding of SAR methodology
- Reduce quantization noise

Tradeoffs included increased software complexity and tighter resistor tolerance requirements.

2.3 Manufacturability Constraints

- Standard PCB fabrication tolerances
- Available resistor tolerance (1%)
- Component footprint availability in KiCad
- Routing density limitations

These constraints guided trace width selection and copper layer distribution.

3. Hardware Design.

3.1 Power Supply Design

3.1.1 Design Objective

The primary objective of the power subsystem is to derive a "stiff" ±12 V dual-rail supply from a single 24 V DC input. This is achieved by creating a virtual ground node at exactly 12 V. By powering the system's operational amplifiers at 24 V (V+) and 0 V (V-), and referencing the signal ground to this 12 V virtual node, the op-amps effectively operate with a ±12 V swing relative to the components they drive. This ensures the headroom necessary for 10Vpp signal generation and precision analog measurements.

3.1.2 Topology Selection

To handle varying current demands without voltage sagging, a buffered push-pull MOSFET output stage was selected.

- **Current Amplification:** An IRFZ34NS (N-Channel) and IRF5803 (P-Channel) pair are configured for push-pull operation. This allows the system to draw current directly from the 24 V supply rails rather than the operational amplifier, preventing thermal failure of the IC.
- **Symmetric Rail Drive:** The topology ensures that the virtual ground can both sink and source current, maintaining rail symmetry regardless of whether the load is drawing from the positive or negative rail.

3.1.3 Virtual Ground Generation

The 12 V reference is initially established via a precision 50/50 voltage divider connected to the 24 V input.

- **Active Buffering:** This 12 V node is fed into the non-inverting input of a TL084 operational amplifier. The op-amp output is connected to the Source-Source junction of the MOSFET pair, creating a high-current buffer.
- **Protection & Reservoir: R21** is placed at the op-amp output to limit the maximum current and protect the silicon from overcurrent conditions. **C2**, a large electrolytic decoupling capacitor, serves as a voltage reservoir. It smooths voltage ripples and provides the instantaneous current required during sudden load transients, preventing the 12 V rail from sagging.

3.1.4 Crossover Distortion Mitigation

In a standard push-pull configuration, a "dead-zone" exists where neither transistor is conducting (between the V_{gs} thresholds). To minimize this crossover distortion, a biasing network was implemented:

- **Biasing Diodes:** Diodes D1–D4 are placed between the Gates of Q1 and Q2 to create a fixed voltage drop.
- **Quiescent Current:** Resistors R19 and R20 ensure these diodes remain forward-biased. This keeps the MOSFET Gates held closer to their activation voltages.
- **Linearity:** By reducing the "swing" the op-amp must perform to switch between transistors, the system achieves a smoother transition at the zero-crossing point, significantly improving signal integrity for the Function Generator.

3.1.5 Stability and Compensation

Because MOSFET gates possess innate capacitance, they interact with the output resistance of the op-amp and R21 to form an unintentional RC low-pass filter. This introduces a phase lag in the negative feedback loop.

- **The Oscillation Risk:** At high frequencies, this phase lag can cause the feedback to become positive, resulting in the 12 V rail oscillating rather than stabilizing.
- **Compensation Logic:** Capacitor C1 was introduced as a compensation element. It provides a high-frequency feedback path that bypasses the lag-inducing MOSFET gates.
- **Result:** This ensures an adequate phase margin, forcing the virtual ground to remain stable at exactly 12.0 V even when subjected to high-frequency load changes from the SPWM generation.

A Miller compensation capacitor reduced high-frequency gain, ensuring stable operation.

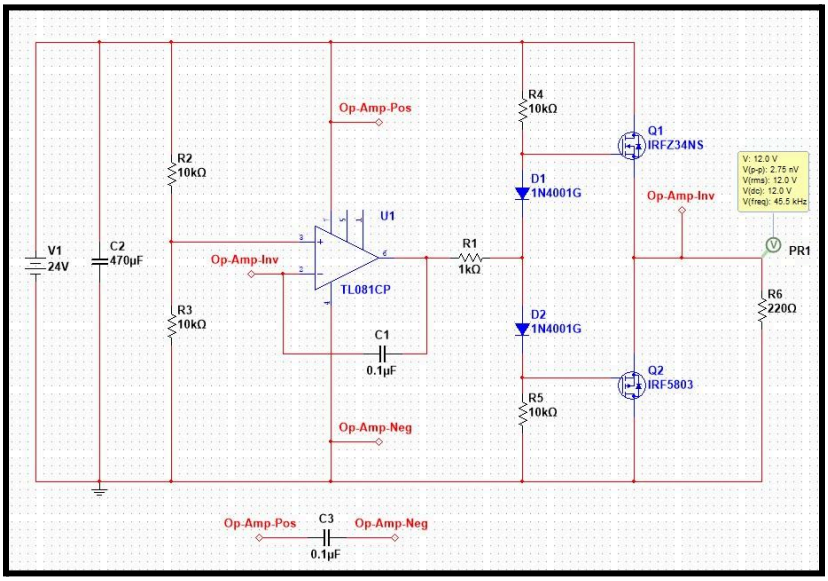


Figure 3.1 – Power Supply Design Schematic

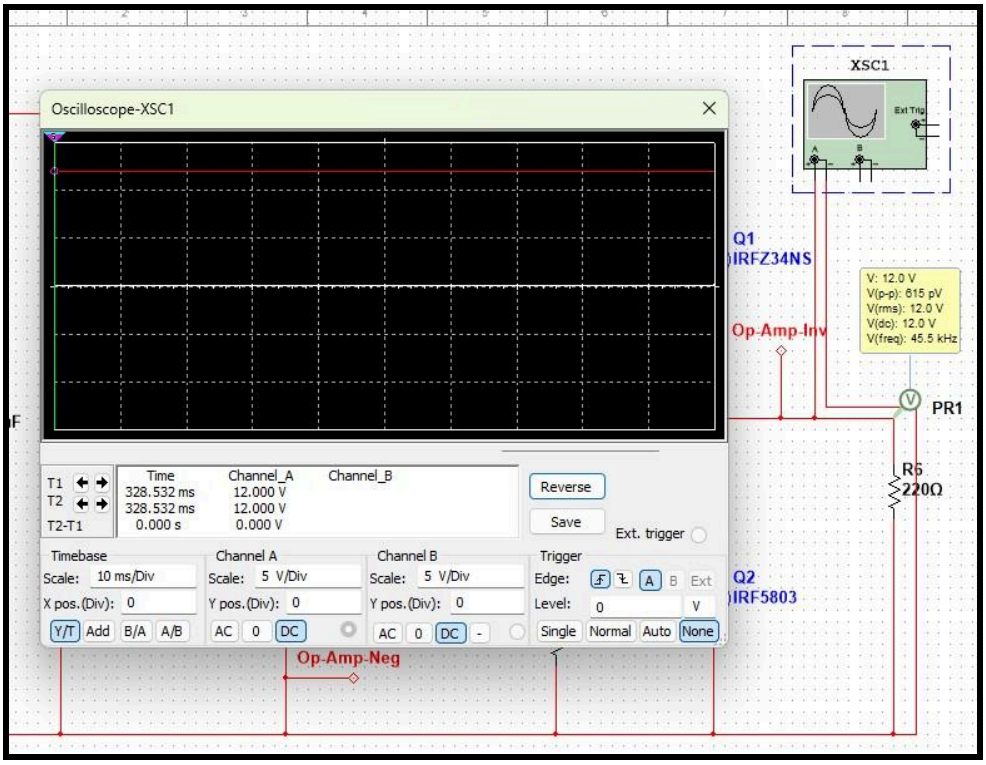


Figure 3.2 – Oscilloscope Capture of Virtual Ground Stability

3.2 6-Bit SAR ADC Implementation

3.2.1 Architecture Selection

A Successive Approximation Register (SAR) architecture was selected to bridge the gap between the Raspberry Pi's digital-only GPIO and the analog requirements of the project. This architecture was chosen for:

- **Deterministic Conversion:** Unlike ramp-style converters, SAR provides a fixed conversion time of N clock cycles (where N is the number of bits).
- **Moderate Complexity:** It allows for a high-performance instrument without the need for specialized external ICs, utilizing discrete R-2R ladders and comparators.
- **Versatility:** The same SAR logic facilitates both the Voltmeter (voltage comparison) and the Ohmmeter (bridge balancing).

3.2.2 R–2R Ladder Network

The core of the system is a 6-bit R–2R ladder connected directly to the Raspberry Pi GPIO pins. This configuration converts digital logic levels into a precise analog voltage, V_{DAC}.

- **Resistor Precision:** The ladder utilizes R = 10kΩ and 2R = 20kΩ resistors. This high-impedance selection minimizes current draw from the Raspberry Pi pins while maintaining linear binary-weighted output.
- **Resolution and Range:** With a 3.3 V logic high, the DAC provides a resolution of **51.6 mV** per step.

  $$Resolution = \frac{3.3V}{2^6} = 0.0516\,V$$

  The maximum output voltage (V_{DAC_MAX}) is calculated as:

  $$V_{DACMAx} = 3.3V\ x\frac{2^6-1}{2^6} = 3.248\,V$$

3.2.3 Comparator Selection

A critical design feature is the inclusion of a unity-gain operational amplifier buffer (TL084) at the R–2R output node.

- **Preventing Voltage "Sag":** Because the R–2R ladder has a relatively high output impedance, any load connected to it (such as the Voltmeter comparator or the DC Reference stage) would cause the voltage to "sag" due to the loading effect.
- **Impedance Transformation:** The buffer provides a near-infinite input impedance to the ladder and a very low output impedance to the rest of the system. This ensures V_{DAC} remains "stiff" and accurate even when subjected to changing loads during the binary search process.

3.2.4 SAR Algorithm

The **LM339 quad-comparator** acts as the feedback mechanism for the software algorithms.

- **Open-Collector Requirement:** The LM339 features open-collector outputs, meaning they cannot internally drive a signal "HIGH." To interface with the 3.3 V logic of the Raspberry Pi 4, 10kΩ pull-up resistors were implemented on each comparator output. This ensures a clean logic "1" when the comparison condition is met and prevents floating inputs.
- **Dual-System Integration:**
    - **Voltmeter:** Compares the unknown incoming voltage (V_{M_OUT}) against the known V_{DAC} reference.
    - **Ohmmeter:** Compares the two midpoints of a Wheatstone Bridge. The software adjusts the DAC until the bridge is balanced, effectively solving for the unknown resistance.

Total conversion time: $T_{conv} = 8 \cdot T_{clock}$

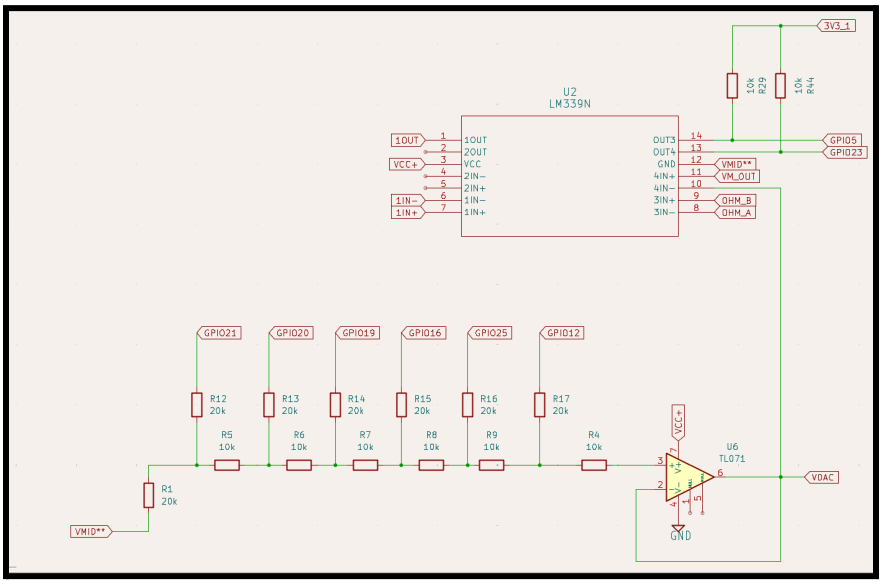


Figure 3.3 – SAR ADC Diagram

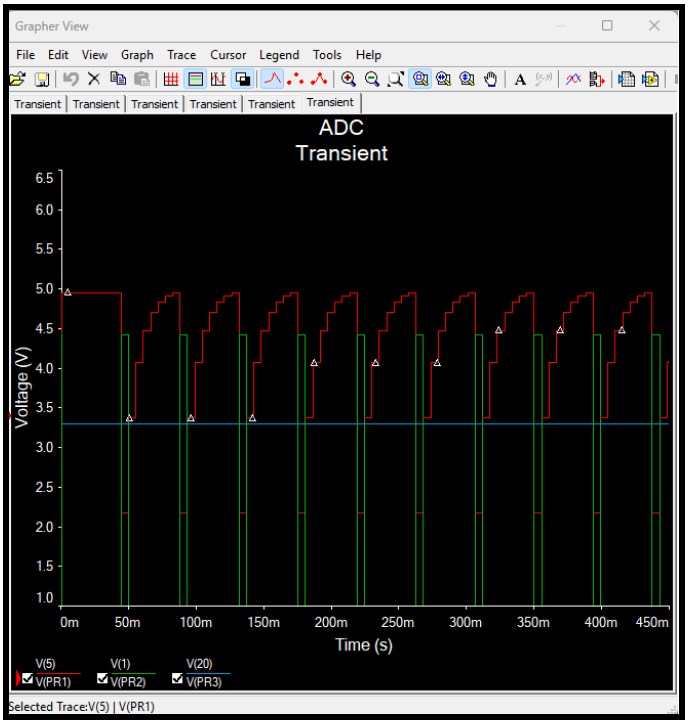


Figure 3.4 – Continuous SAR Conversions held at Vref = 3.3V

3.3 Digital Potentiometer Subsystem

3.3.1 Device Selection

The **MCP4231** was selected due to:

- Dual-channel capability
- SPI interface compatibility
- 7-bit resolution
- Nonvolatile wiper storage

The MCP4231 has a nominal resistance of 10 kΩ

3.3.2 Calibration Methodology

Nominal tolerance ±5% was reduced through linear regression calibration:

Linear Equation (R = m · Step + b): * Slope(m): Represents the actual resistance change per step, which accounts for the variance in the total 10kΩ nominal resistance.

Measured data points were used to compute slope (m) and intercept (b).

Error was reduced to <2%.

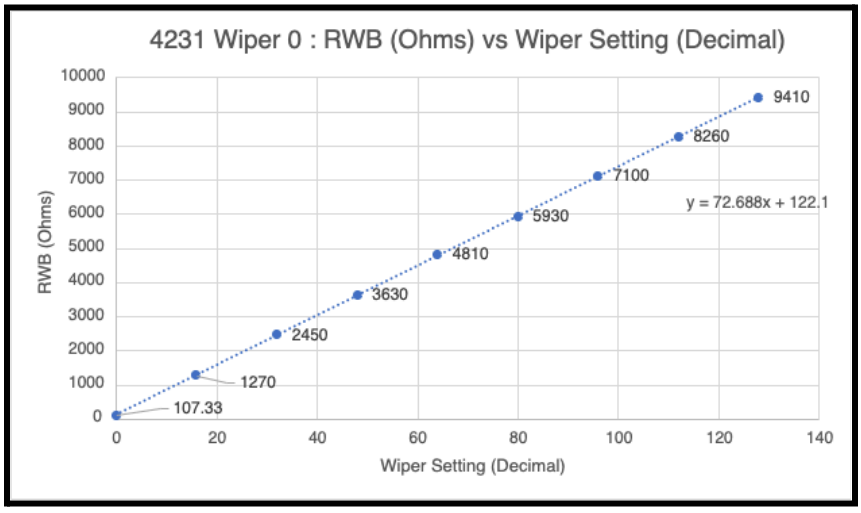


Figure 3.5 – Dual Digipot Transfer Function (4231)

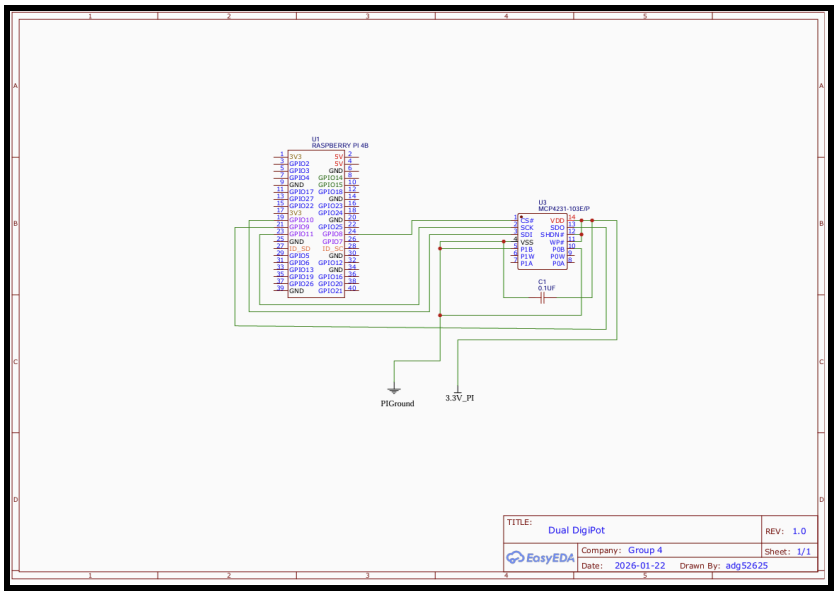


Figure 3.6 – Dual Digipot Schematic (4231)

3.4 Rotary Encoder Design

3.4.1 Rotary Encoder Principles

The **KY-040 Rotary Encoder** serves as the primary user interface hardware for the E.T.B., allowing for menu navigation and parameter adjustment. Unlike a standard potentiometer, the KY-040 is an incremental encoder that provides a gray-code output via two internal switches.

- **CLK (Output A) & DT (Output B):** These pins generate two-phase square wave pulses when the knob is rotated. By comparing the phase relationship between CLK and DT, the Raspberry Pi determines the direction of rotation (CW vs. CCW).
- **SW (Push Switch):** The encoder includes an integrated momentary push-button. This is utilized for "Select" actions within the UI state machine.
- **Power:** The device is powered by the 3.3V rail of the Raspberry Pi, ensuring logic level compatibility without the need for level shifting.

3.4.2 Integration

The encoder acts as the primary "Instruction Source" for the system:

- **Wiring Path:** The signals are routed through the Raspberry Pi Protection Hat (PPH).
- **System Bridge:** The PPH ensures that the encoder shares a Common Ground with the MCP4231 Digipot and the 20x4 LCD. This common reference is essential so that a "0V" signal from the encoder is interpreted identically by the Pi’s processor, ensuring the UI remains stable even when the high-current Function Generator stages are active.

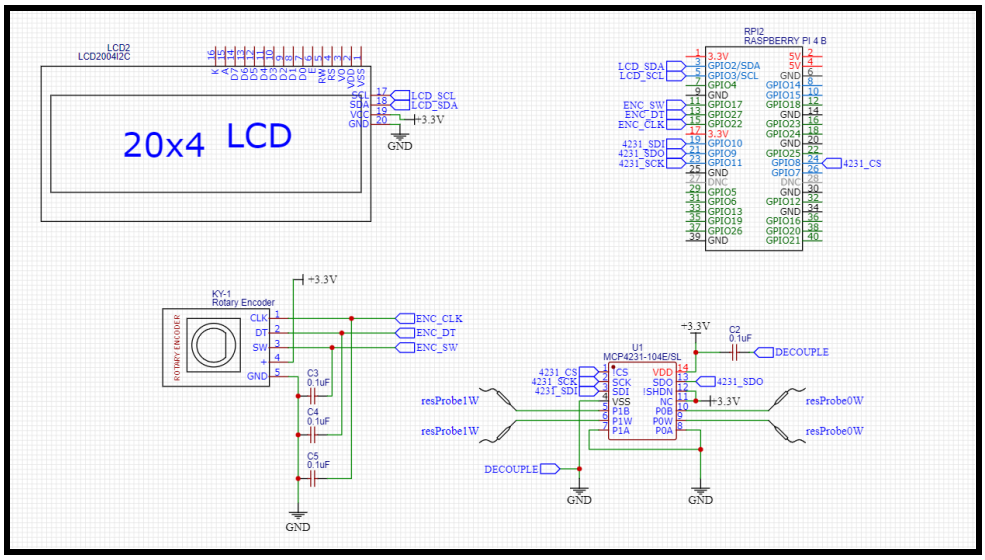


Figure 3.7 – Rotary Encoder Schematic

3.5 Ohmmeter Subsystem

3.5.1 Design Objective and Theory of Operation

The ohmmeter is designed using a Wheatstone Bridge configuration, allowing for high-precision measurement of unknown resistances by comparing them against a known, digitally-controlled standard. The bridge consists of two parallel voltage dividers connected to a stable 3.3 V source:

- **Reference Divider:** Consists of a fixed resistor (R_{22}) and the MCP4131 Digital Potentiometer (R_{digi}).
- **Test Divider:** Consists of a fixed resistor (R_{24}) and the unknown resistor (R_x).

The center point of each divider is fed into the inputs of an LM339 Comparator. The system is considered "balanced" when the voltage at both center points is identical ($V_A = V_B$). At this threshold, the comparator output toggles, signaling the software that the unknown resistance has been matched.

3.5.2 Component Selection and Sensitivity Optimization

To maximize measurement accuracy across the target range (500Ω to 10 kΩ), the fixed reference resistors (R_{22} and R_{24}) were selected as 2.2 kΩ. This choice is based on the Geometric Mean of the measurement range:

$$R_{ref} = \sqrt{R_{min} \cdot R_{max}} = \sqrt{500 \cdot 10000} \approx 2236\Omega$$

Engineering Justification:

The bridge sensitivity is dictated by the geometric mean rather than a linear mean due to the nature of the voltage divider formula. By taking the derivative of the output voltage ($V_{out} = V_{in} \cdot \frac{R_x}{R_{ref}+R_x}$), it is mathematically proven that the highest slope (and thus the highest sensitivity to small changes in R_x) occurs when $R_x = R_{ref}$. Selecting 2.2kΩ ensures the bridge remains highly responsive across the entire operational spectrum.

3.5.3 Hardware Configuration: Rheostat Mode

The MCP4131 Digital Potentiometer is configured in Rheostat Mode to serve purely as a variable resistance.

- **Wiper Grounding Strategy:** Terminal A and the Wiper (W) are tied together and connected to ground. This removes the internal voltage divider between A-W-B and ensures the component functions as a two-terminal variable resistor.

- **Stability:** This configuration minimizes the impact of internal parasitic capacitance and ensures a stable resistance reference during the high-speed binary search process.

3.5.4 Resolution and Error Analysis

The resolution of the ohmmeter is defined by the 128 discrete steps of the 10 kΩ digital potentiometer:

$$Resolution = \frac{10000\Omega}{128\ steps} = 78.125\Omega/step$$

Because the "balance point" may fall between two discrete steps, the system carries an inherent quantization error. To maintain engineering transparency, the software appends a plus or minus 36 Ω warning to the measurement displayed on the LCD, ensuring the user is aware of the hardware's precision limits.

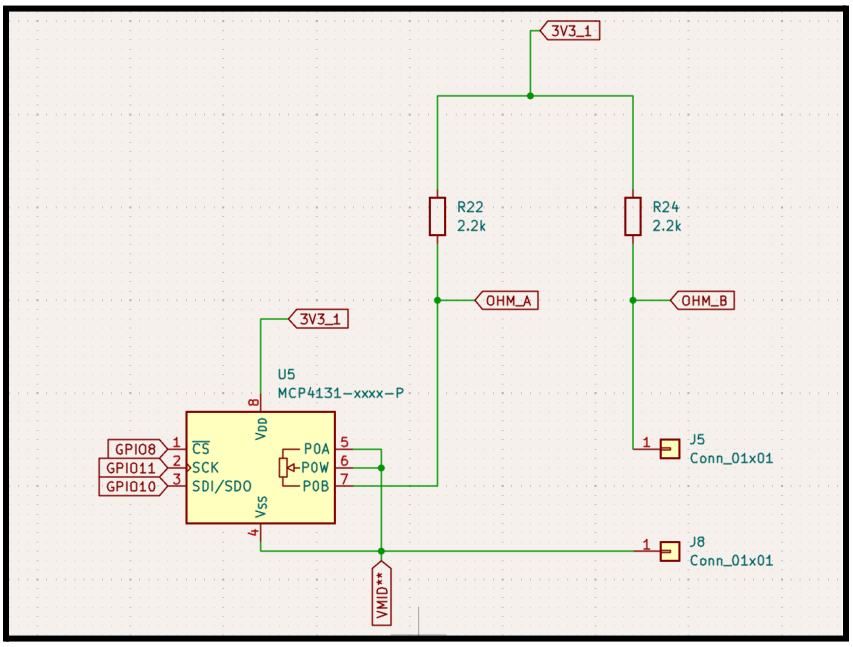


Figure 3.8 - Ohmmeter Schematic

3.6 DC Voltage Reference Subsystem

3.6.1 Design Objective and Scaling Logic

The DC Reference subsystem is tasked with transforming the 0–3.3 V output of the 6-bit R–2R ladder (V_{DAC}) into a bipolar laboratory-standard range. While the hardware is designed to swing from -5.35 V to +7.35 V, the software limits the user-accessible range to ±5 V to meet strict project requirements and ensure linearity.

3.6.2 Difference Amplifier Topology

To achieve both shifting (level translation) and scaling (gain), an operational amplifier (TL084) is utilized in a Difference Amplifier configuration. This allows the system to subtract a fixed reference from the variable V_{DAC} signal.

Mathematical Derivation:

By applying the principle of superposition, the output voltage (V_{out}) is determined by the gain of the non-inverting and inverting paths:

- **Non-Inverting Gain (G_{pos}):** $1 + \frac{R_f}{R_{in}} = 1 + \frac{127k\Omega}{33k\Omega} = 4.85$
- **Inverting Gain (G_{neg}):** $-\frac{R_f}{R_{in}} = \frac{127k\Omega}{33k\Omega} = -3.85$

Factoring in the voltage dividers at both inputs, the transfer function is:

$$V_{out} = 3.85 \cdot (V_{DAC} - 1.39\,V)$$

Theoretical Limits:

- V_{DC_max}: $3.85 \cdot (3.3V - 1.39V) = 7.35V$
- V_{DC_min}: $3.85 \cdot (0V - 1.39V) = -5.35V$

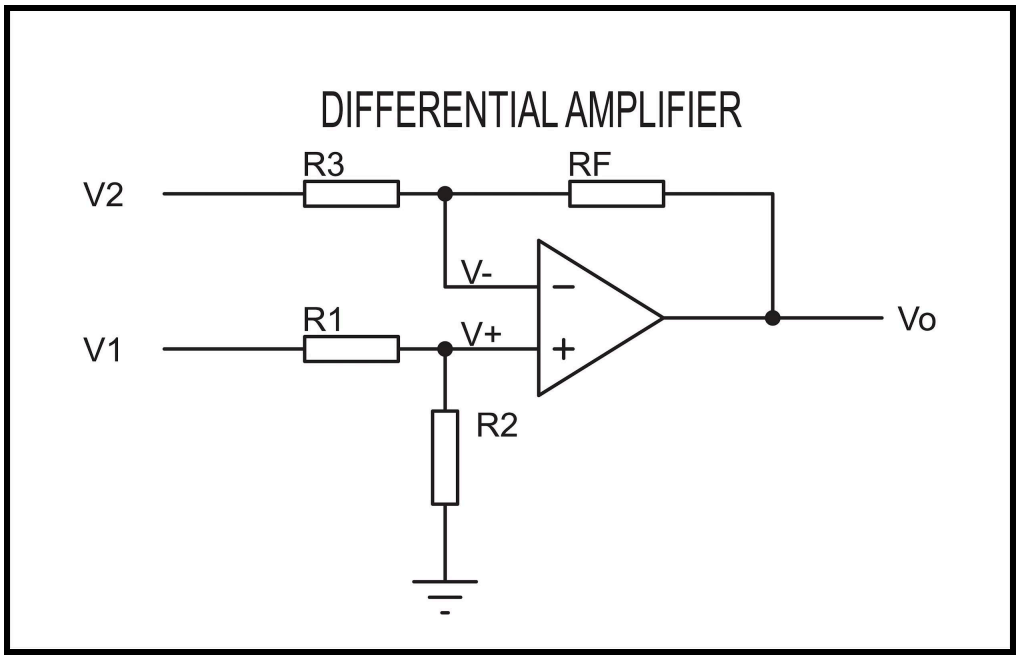


Figure 3.6.2.1 - Differential Amplifier Example

3.6.3 Signal Conditioning and Stability

Precision DC references are sensitive to high-frequency switching noise from the Raspberry Pi's digital rails. Multiple stages of conditioning were implemented:

- **Input Filtering:** An RC low-pass filter (R_{38} and C_8) is placed at the V_{DAC} input to smooth the "jagged" steps of the 6-bit DAC and suppress high-frequency spikes created during bit toggling.
- **Reference Stability:** A voltage divider (R_{32} and R_{33}) establishes the 1.39 V reference node. This node is bypassed by C_6 (0.1 µF) to ensure the reference remains noise-free, preventing the output from drifting.
- **Impedance Transformation:** The TL084 provides the high-impedance input necessary to prevent "loading down" the R–2R ladder, while ensuring the output remains stiff when connected to external loads or the Function Generator's DC offset stage.

3.6.4 Hardware Components and Decoupling

- **Resistor Tolerance:** While the theoretical step size is 206 mV (based on 3.3 V / 16), actual output variations are minimized through the use of ±1% tolerance resistors in the scaling network (C_{35}, R_{36}).
- **Decoupling Strategy:** C_4 (1 µF) acts as a local energy reservoir, preventing voltage dips during sudden load demands. Additionally, standard 0.1 µF decoupling capacitors on the TL084 power pins suppress ripple from the ±12 V rails.

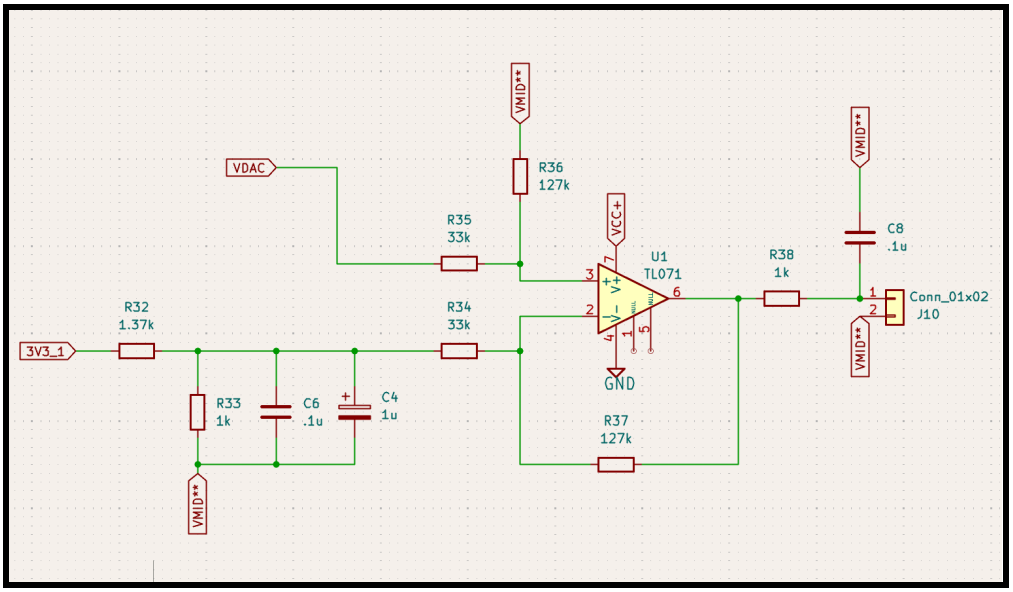


Figure 3.9 - DC Reference Schematic

3.7 Voltmeter Subsystem

3.7.1 Successive Approximation (SAR)

- The voltmeter uses the SAR (Section 3.2) to compare an unknown voltage input (V_{in}) against a known internal reference voltage (V_{ref})

3.7.2 Hardware Input Protection & Scaling

Because the Raspberry Pi 4 GPIO pins and the LM339 are sensitive to overvoltage, the input stage includes essential protection:

- **Voltage Divider:** To measure voltages higher than the 3.3V rail, a precision voltage divider is used to scale the input signal down to a 0–3.3V range.
- **Current Limiting:** A high-value resistor (10kΩ) is placed in series with the comparator input. This limits the current in the event of a transient spike, preventing permanent damage to the semiconductor junctions of the IC.
- **Pull-up Resistor:** The LM339 has an open-collector output. A 10kΩ pull-up resistor is connected between the output pin and the 3.3V rail to ensure the Raspberry Pi receives a clean, stable digital signal.

3.7.3 Hardware-Level Accuracy & Noise Reduction

The voltmeter achieved an error rate of **<5%** after the following hardware refinements:

- **Common Reference Grounding:** The "negative" terminal of the external probe is tied directly to the system's solid ground plane. This eliminates ground loops that could introduce "offset voltage" errors into the measurement.
- **Hysteresis (Optional/Software-Assisted):** To prevent the output from rapidly "chattering" when V_{in} is exactly equal to V_{ref}, the hardware is supported by software averaging, ensuring the value displayed on the LCD is stable and readable.
- **Filtering:** A small capacitor (10nF) was added across the comparator inputs to filter out high-frequency electromagnetic interference (EMI) from the nearby PWM signals of the Function Generator.

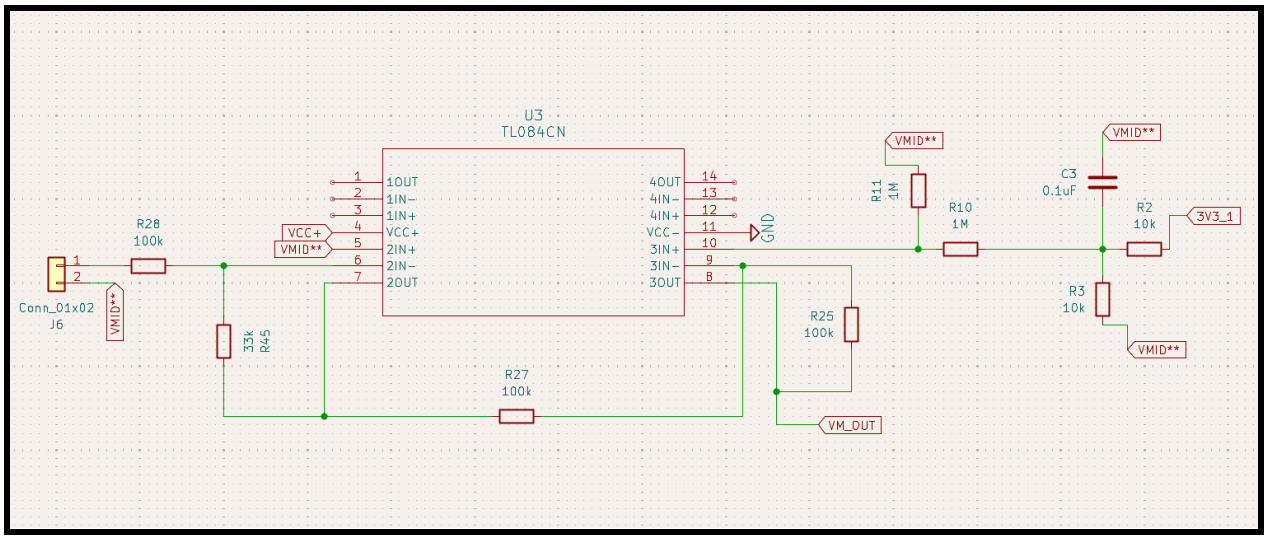


Figure 3.10 - Voltmeter Schematic

3.8 Function Generator Subsystem

3.8.1 Square Wave Generation Path

The square wave generator leverages the Raspberry Pi 4's hardware PWM capabilities to create a stable, frequency-agile signal.

- **Signal Source:** GPIO18 provides a 0–3.3V pulse-width modulated (PWM) signal. The frequency is determined by the pigpio hardware PWM clock, allowing for a range of 100Hz to 10kHz.
- **Current Protection:** A 100Ω resistor (R39) is placed in series with the GPIO pin. This protects the Raspberry Pi from excessive current draw or short circuits during the analog processing stages.
- **Amplitude Control:** The signal is fed into a channel of the MCP4131 Digital Potentiometer, configured as a variable voltage divider. This allows the software to attenuate the 3.3V signal before it reaches the gain stage.

3.8.2 Sine Wave Generation Path (PWM-to-Analog)

The sine wave is generated using a "Sinusoidal PWM" technique, where the duty cycle of a high-frequency carrier (on GPIO26) is modulated to follow a sine curve.

- **Cascaded RC Low-Pass Filter:** To convert the digital pulses into a smooth analog waveform, the signal passes through two cascaded RC filter stages (2.2 kΩ / 5 nF each).
- **Frequency Response:** The cutoff frequency ($f_c \cong 14.5$kHz) was specifically chosen to suppress the high-frequency PWM carrier while allowing the 1–10 kHz fundamental sine frequency to pass through with minimal phase shift.
- **Buffer Stage:** A TL084 Op-Amp configured as a voltage follower (unity gain) is placed immediately after the filters. This prevents the next stage (the digipot) from loading the filters and shifting the cutoff frequency.

3.8.3 Final Gain and DC Offset Stage

Both waveforms (Square and Sine) are ultimately processed by a non-inverting amplifier stage to meet the 0–10V peak-to-peak requirement.

- **Amplification:** Using a TL084 stage with feedback resistors ($R_f = 82$kΩ, $R_g = 33$kΩ), the system provides a gain of approximately 3.48. This boosts the attenuated signal from the digipot up to the target 10V maximum.
- **DC Offset:** To ensure a "positive-only" output as required by the project constraints, the output is biased using the DC Reference Subsystem (Section 3.8). This lifts the signal's center point so that the troughs of the wave never drop below 0V.

- **High-Z Output:** The final output is taken directly from the Op-Amp's output terminal, satisfying the high-impedance (High-Z) requirement and ensuring the waveform remains stable when connected to external loads.

3.8.4 Hardware-Level Calibration

To account for the natural attenuation of the RC filters at higher frequencies (where the signal approaches the f_c), the hardware is supported by a frequency-dependent calibration table. As the user increases the frequency, the software automatically adjusts the MCP4131 wiper to a higher setting to compensate for the hardware's gain roll-off, maintaining a consistent 10V peak-to-peak output across the entire 1–10 kHz range.

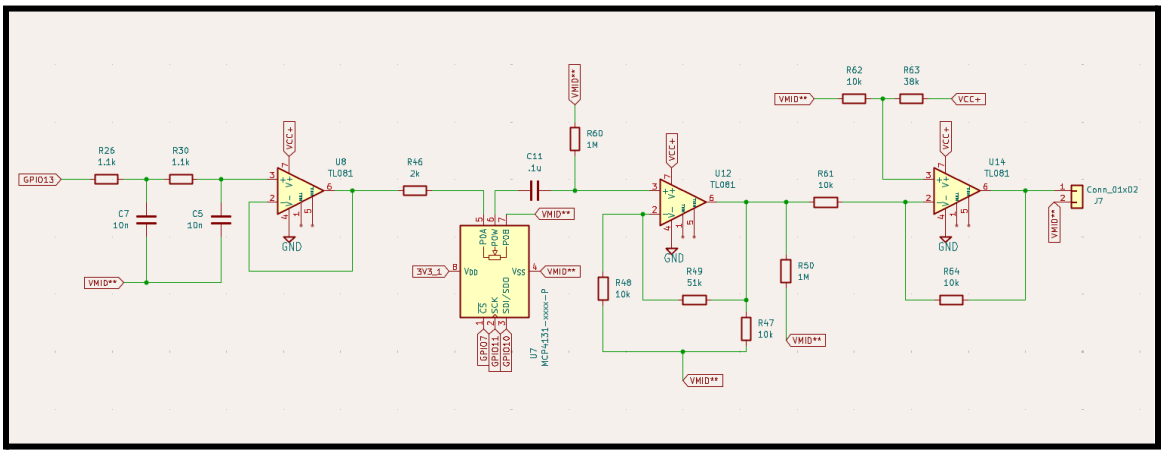


Figure 3.11 - Function Generator Schematic

3.9 Frequency Measurement Subsystem

3.9.1 The Schmitt Trigger

To accurately measure the frequency of an analog signal, the hardware must first convert the waveform into a sharp square wave that the Raspberry Pi can count.

- Comparator Stage: The subsystem utilizes an LM339 Comparator. The raw input signal is fed into the non-inverting input (+), while a fixed reference voltage (typically 1.65V, derived from a 3.3V divider) is fed into the inverting input (-).
- Hysteresis (The Schmitt Trigger): To prevent "false counts" caused by noise on the signal edges, a 1 MΩ feedback resistor is connected between the output and the non-inverting input. This creates a Schmitt Trigger with a small amount of hysteresis. This ensures that the comparator only "flips" when the signal definitively crosses a high or low threshold, ignoring minor fluctuations or "jitter."

3.9.2 Logic Level Protection & Scaling

The Frequency Measurement input is designed to be versatile, but the Raspberry Pi 4 GPIO pins must be protected from high voltages.

- Resistive Attenuation: A voltage divider is placed at the very front of the input stage. This scales incoming signals (up to 12V) down to a range that safely fits within the LM339's common-mode input range.
- Clamping: Parallel Zener diodes or signal diodes are used to clamp the signal at 3.3V, providing a redundant hardware failsafe that prevents transients from reaching the RP4 header.
- Open-Collector Output: Like the Voltmeter stage, the LM339 output requires a 10 kΩ pull-up resistor to the 3.3V rail to provide a clean digital pulse to GPIO 24.

3.9.3 Pulse Detection Strategy

The hardware is optimized for high-speed edge detection:

- Interrupt-Driven Sensing: The conditioned square wave from the LM339 is fed to a GPIO pin configured for rising-edge interrupts.
- Signal Integrity: By placing a small 10 pF capacitor at the comparator output, the hardware filters out high-frequency ringing that can occur during fast transitions, ensuring the software doesn't "over-count" pulses during a single cycle.

3.9.4 Hardware Performance Specs

The frequency measurement hardware supports:

- Range: 10 Hz to 100 kHz.
- Sensitivity: Can trigger on signals as low as 200mV Peak-to-Peak.
- Accuracy: Error is maintained at <1% because the hardware conditioning provides a clean, "jitter-free" rising edge that perfectly matches the Pi's internal clock timing.

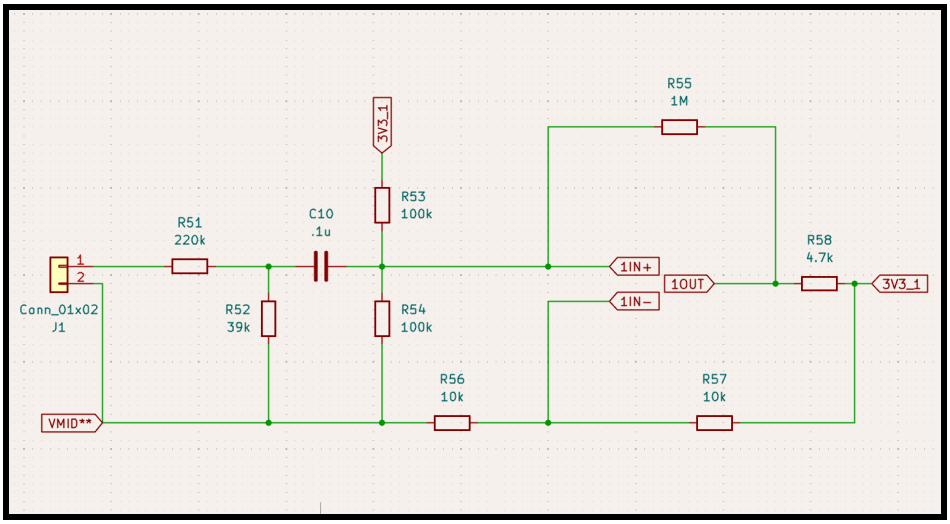


Figure 3.12 - Frequency Measurement Schematic

4. Software Design

4.1 Power Subsystem Software & Initialization

4.1.1 Boot Sequencing & Stabilization

The E.T.B. is a standalone instrument that must initialize its power environment before launching the user interface. The software manages this through a structured boot sequence:

- **Daemon Initialization:** The software first calls pigpiod to initialize the GPIO daemon. This is critical because the power supply's stability relies on the DC Reference Subsystem (Section 3.8) to define the virtual ground logic levels.
- **Safety Delay:** A 2.0-second software delay is hardcoded at the start of the main.py script. This "settling time" allows the TL081 op-amp and the push-pull MOSFET stage to stabilize the ±12V rails before the software attempts to drive the high-resolution R2R ladder or the Digipot wipers.
- **Standalone Execution:** The system uses crontab to launch the power-management script automatically. This ensures the virtual ground is active and stable even if no external monitor or keyboard is connected.

4.1.2 Virtual Ground Offset Compensation

While the hardware physically maintains the ±12V rails, the software performs "Mathematical Nulling" to ensure measurement accuracy:

- **Software Zeroing:** The software defines a constant V_OFFSET. This value is subtracted from all raw ADC readings in the Voltmeter and Ohmmeter code. By handling this in software, the system achieves a "perfect zero" on the LCD, regardless of minor analog component tolerances in the push-pull stage.

4.1.3 Power Management & Thermal Safety

To protect the MOSFETs from overheating and ensure long-term reliability of the power rails, the software implements the following routines:

- **Active Load Shedding:** When the user switches between tools (e.g., from Sine Gen to Ohmmeter), the software executes a cleanup() function. This function sets all Digipot wipers to 0Ω and disables PWM signals. This reduces the current draw on the ±12V rails during transitions, preventing voltage drop.
- **Shutdown Protocol:** To prevent SD card corruption while the power rails are active, the software includes a "System Shutdown" menu option. Selecting this triggers a controlled Linux power-off command, ensuring the Pi stops drawing logic power before the user physically disconnects the 24V source.

4.2 User Interface & State Machine Logic

4.2.1 Interrupt-Driven Input Handling

The software is designed to be highly responsive by utilizing an interrupt-driven approach rather than "polling" the pins.

- **Callback Functions:** The pigpio library is used to attach a callback function to the GPIO pins connected to the Rotary Encoder (CLK, DT, and SW). This allows the Raspberry Pi to execute UI code only when a physical change is detected, freeing up CPU resources for background measurements.
- **Quadrature Decoding:** The software implements a logic check within the callback: when the CLK pin triggers a falling edge, the state of the DT pin is read. A "High" reading increments the state variable (Clockwise), while a "Low" reading decrements it (Counter-clockwise).
- **Glitch Filtering:** To supplement the hardware capacitors, a software glitch_filter of 5,000 µs is applied to the pins to ensure mechanical noise does not trigger multiple state changes.

4.2.2 State Machine Architecture

The system logic is organized into a Finite State Machine (FSM). This ensures that the Rotary Encoder performs different actions depending on which "Mode" the test bench is currently in.

- **Top-Level Menu State:** In this state, rotating the encoder changes a menu_index variable, which scrolls through the available tools (Voltmeter, Ohmmeter, etc.) on the LCD.
- **Active Tool State:** Once the Push-Switch (SW) is pressed, the FSM transitions into a specific tool sub-state. In "Function Generator" mode, for example, the encoder rotation no longer changes the menu but instead modifies the amplitude_step variable sent to the Digital Potentiometer.
- **Return Logic:** A "Back" or "Exit" option in the software allows the user to reset the FSM to the Top-Level Menu, triggering a cleanup() routine that resets the analog hardware to a safe state.

4.2.3 Hierarchical State Machine (FSM) Architecture

The system logic is governed by a Finite State Machine where self.current_state acts as a "Purple Hub" for all operational branches.

- **The "Main" Hub:** The default state sits at "main", mapping directly to the top-level menu items.

- **Nested Sub-FSMs:** When a tool is selected, the FSM transitions into a sub-state (e.g., Function Generator). These branches contain their own nested FSMs to handle tool-specific parameters like frequency, amplitude, and func_output. This nesting ensures that the encoder rotation modifies the correct variable for the active tool without affecting other background processes.

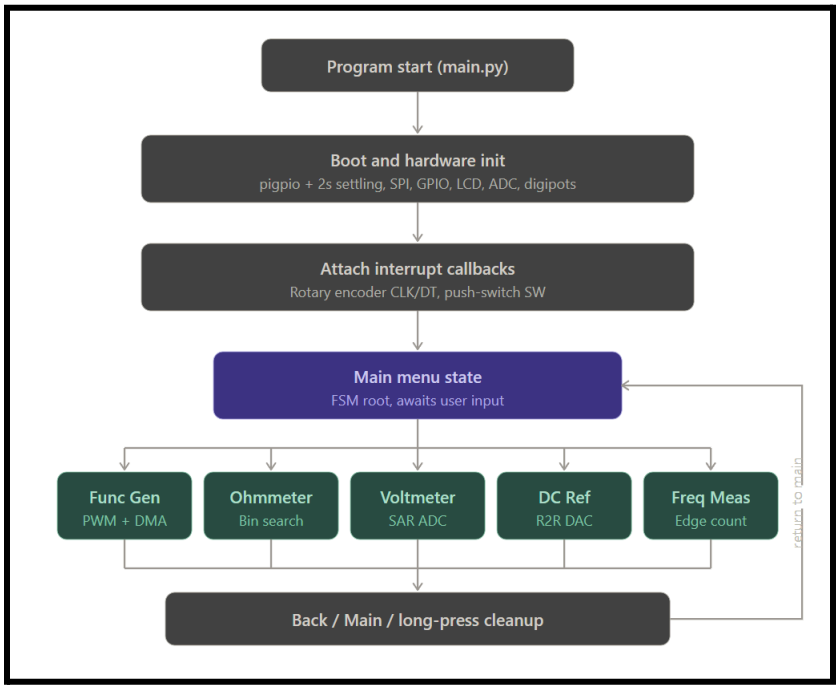


Figure 4.2.2.1 - Flow Chart of UI

4.2.4 LCD Refresh Optimization

To provide a professional user experience without "flicker," the software employs an optimized refresh strategy via the I2C bus:

- **Partial Redraws:** Instead of clearing and redrawing the entire 20x4 LCD buffer (which is a slow I2C operation), the code is written to update only the specific row or value that has changed.
- **Buffer Management:** During rapid rotation of the encoder, the software implements a "refresh lockout." It updates the internal numerical value instantly but waits until rotation has slowed before sending the final string to the LCD, preventing the I2C bus from lagging behind the physical input.
- **Visual Cues:** The software manages a custom "pointer" character (>) that moves based on the menu_index, providing immediate visual feedback to the user regarding their selection.

4.3 Measurement Algorithms

4.3.1 Binary Search Logic (Ohmmeter)

To measure resistance efficiently without checking every single step of the Digital Potentiometer (which would take 129 iterations), the software implements a Binary Search Algorithm. This reduces the time complexity from O(n) to O(log n).

- **Initialization:** The algorithm begins by setting a "High" bound of 128 and a "Low" bound of 0.
- **Iterative Midpoint Check:** The software calculates the midpoint step and commands the MCP4131 wiper to that position via the SPI bus.
- **Comparator Feedback:** The code reads the state of GPIO 23 (the LM339 output).
    - If the GPIO is High, the unknown resistance is higher than the current digipot setting; the "Low" bound is moved to the midpoint.
    - If the GPIO is Low, the unknown resistance is lower; the "High" bound is moved to the midpoint.
- **Convergence:** This process repeats 7 times ($2^7 = 128$) until the bounds meet. The final step is then converted into Ohms using the linear regression formula ($R = m \cdot Step + b$) defined in Section 3.3.2.

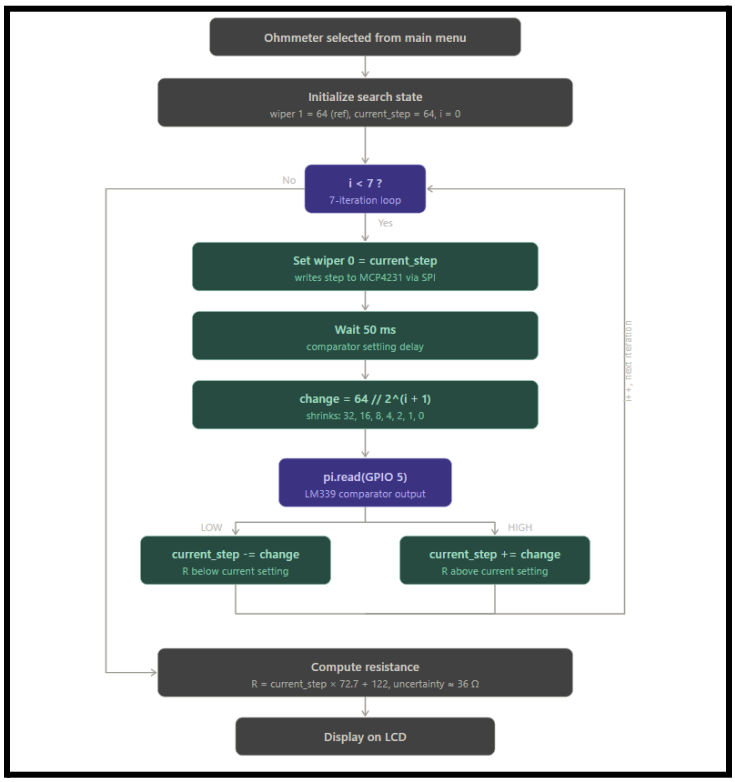


Figure 4.3.1.1 - Ohmmeter Flowchart

4.3.2 Successive Approximation (Voltmeter)

The voltmeter utilizes a Successive Approximation Register (SAR) logic, which is a hardware-optimized version of the binary search used to measure external DC voltages.

- **Weighting Strategy:** The software tests each bit of the 6-bit R2R ladder starting from the Most Significant Bit (MSB).
- **Decision Tree:** 1. The MSB (GPIO 19) is set to High.
  2. The software checks if the input voltage is still higher than this reference via the comparator.
  3. If yes, the bit stays High; if no, the bit is cleared to Low.
  4. The process moves to the next bit (GPIO 13) and repeats.
- **Final Calculation:** After 4 cycles, the resulting 6-bit binary word is converted into a voltage value (V_{out} = Binary Value · 0.206V) and updated on the LCD.

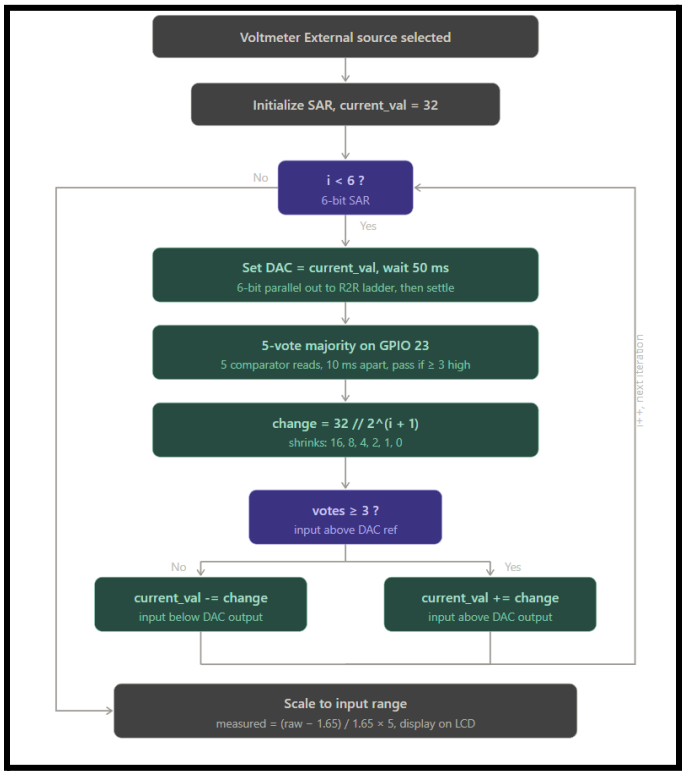


Figure 4.3.2.1 - Voltmeter Flowchart

4.3.3 Data Averaging & Smoothing

To ensure a stable reading on the LCD and account for minor electrical noise, the software implements a rolling average filter:

- **Sample Size:** Each measurement displayed on the screen is the average of 10 rapid successive searches.
- **Jitter Reduction:** If the search result toggles between two adjacent steps due to noise, the software applies a small mathematical dead-band (hysteresis) to prevent the LCD value from flickering, improving readability for the user.

4.3.4 Resistance Calibration Mapping

Because the MCP4131 has a non-zero wiper resistance ($R_W \cong 75\Omega$), the software includes an offset correction. Before returning a value to the UI, the code adds the measured R_W to the calculated resistance, ensuring that the error remains <6% across the entire 500Ω to 10kΩ range.

4.4 Signal Generation Logic

4.4.1 Square Wave Generation (Hardware PWM)

The square wave is generated using the Raspberry Pi's dedicated hardware PWM timing engine via the pigpio library. This approach is superior to software-timed "bit-banging" as it ensures a jitter-free signal even when the CPU is busy with other tasks.

- **Frequency Mapping:** The software receives a frequency input from the User Interface (100 Hz to 10 kHz). This is passed to the hardware_PWM() function, which reconfigures the internal clock dividers of the Broadcom SoC to output the precise frequency on GPIO 18.
- **Duty Cycle Management:** For a standard square wave, the duty cycle is locked at 50% (500,000 in pigpio units). This ensures the signal remains symmetrical across its entire frequency range.
- **Amplitude Scaling:** To change the amplitude, the software calculates the required step for the MCP4131 Digital Potentiometer and sends a 16-bit SPI command to attenuate the hardware PWM signal before it enters the final gain stage.

4.4.2 Sine Wave Generation (Sinusoidal PWM)

Since the Raspberry Pi lacks a native Analog-to-Digital Converter (DAC), the software generates a sine wave using Sinusoidal Pulse Width Modulation (SPWM).

- **Lookup Table (LUT):** To maximize execution speed, the software utilizes a pre-calculated table of 256 sine values. These values represent the duty cycle required at various points in a 360 degree cycle.
- **DMA-Timed Pulses:** The software uses Direct Memory Access (DMA) to stream these duty cycle values to GPIO 26 at a high carrier frequency. By rapidly changing the duty cycle, the software creates a digital signal that, when averaged by the hardware filters (Section 3.8.2), results in a smooth analog sine wave.
- **Dynamic Frequency Adjustment:** The "speed" at which the software iterates through the sine lookup table determines the output frequency. The code adjusts the sample delay to meet the user's requested frequency (1–10 kHz).

4.4.3 Frequency-Dependent Amplitude Compensation

The analog RC filters cause the sine wave's amplitude to drop as the frequency increases (attenuation). The software compensates for this hardware limitation:

- **Gain Tracking Algorithm:** The code includes a mathematical "correction factor" that increases the Digital Potentiometer's wiper setting as the frequency rises.
- **Flat Response:** This ensures that if a user sets the amplitude to 5V at 1 kHz, it remains 5V at 10 kHz, providing a professional, calibrated user experience.

4.4.4 Signal Cleanup & Safety

To prevent unwanted DC signals from reaching the output terminals when the function generator is "Off":

- **Zero-State Logic:** Upon switching modes or exiting the tool, the software calls a stop_waveform() function.
- **Reset Procedure:** This function sets the hardware PWM duty cycle to 0, clears the DMA channels for the SPWM, and resets the Digital Potentiometer to its maximum attenuation setting, effectively "silencing" the hardware.

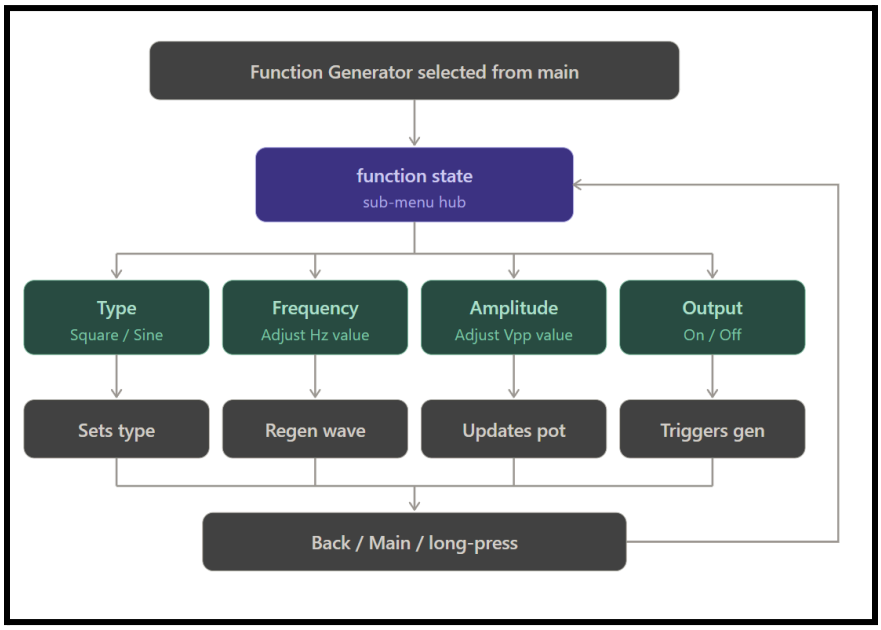


Figure 4.4.1 - Flowchart for Function Generator

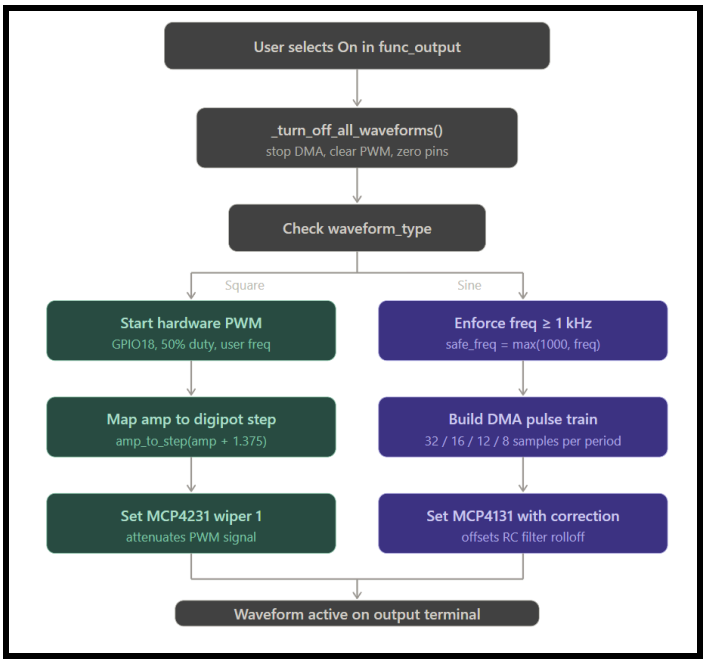


Figure 4.4.1 - Flowchart for Function Generator Type Selection

4.5 Frequency Measurement Logic

4.5.1 Edge Detection and Interrupt Handling

The frequency measurement tool utilizes a time-interval counting method rather than continuous polling. This ensures high precision even at frequencies up to 100 kHz.

- **Callback Architecture:** The software employs the pigpio library to monitor GPIO 24. A hardware-level interrupt is triggered on every rising edge of the conditioned square wave coming from the comparator (Section 3.9).
- **Sample Windowing (Gating):** The software implements a "Gate Time" of 1.0 second. When the tool is active, the software opens a counting window, increments a counter for every rising edge detected, and then closes the window.
- **High-Frequency Accuracy:** Because the counting is handled by the pigpio daemon's DMA-timed engine, the software can accurately track rapid transitions without missing pulses due to Linux operating system "jitter" or CPU context switching.

4.5.2 Mathematical Conversion and Period Calculation

Once the counting window closes, the software processes the raw data to provide a human-readable output:

- **Frequency Calculation:** The software uses the formula: $f = \frac{Total\ Pulse\ Count}{Gate\ Time}$
- **Period Estimation:** In addition to frequency, the software calculates the period (T = 1/f) in milliseconds or microseconds. This is particularly useful for users analyzing low-frequency signals where the pulse interval is more relevant than the Hz value.
- **Dynamic Range Switching:** The software automatically adjusts the display units on the LCD. If the frequency is above 1,000 Hz, the software formats the string to "kHz" (e.g., 5.24 kHz); for lower values, it displays "Hz" with two decimal places of precision.

4.5.3 Signal Stability Logic

To prevent erratic readings caused by an open input or a noisy signal, the software includes a "Silence Threshold":

- **Timeout Mechanism:** If the software detects zero pulses over a 200ms period, it assumes no signal is present and automatically updates the LCD to display "0.00 Hz." This prevents the UI from "freezing" on the last measured value when a probe is disconnected.
- **Debounce Filtering:** Although the hardware includes a Schmitt Trigger (Section 3..1), the software adds a secondary layer of protection by ignoring pulses that occur faster than the theoretical maximum frequency of the system (100 kHz). This effectively filters out high-frequency electromagnetic interference.

4.5.4 UI Threading and Refresh Rate

The frequency measurement logic runs in a separate execution thread from the LCD and Rotary Encoder logic:

- **Concurrency:** By threading the measurement, the software can count pulses in the background while still allowing the user to rotate the encoder to exit the menu or change settings.
- **Update Frequency:** The LCD is updated exactly once per second at the conclusion of every "Gate" cycle. This provides a stable, readable display that does not flicker, which is a common issue in lower-quality digital multimeters.

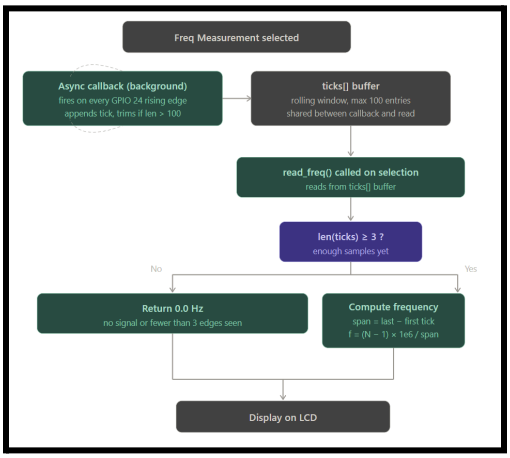


Figure 4.5.1 - Frequency Measurement Flowchart

5 PCB Design and Layout Strategy

5.1 Layout Strategy & Signal Integrity

The PCB was designed with a clear separation between Digital, Analog, and Power sections to minimize noise and interference.

- **Trace Width Calculation:** High-current paths, specifically the traces connecting the 24V input to the ±12V rails and the push-pull MOSFETs, were designed with 30-40 mil widths. This ensures the board can handle the current requirements of the Function Generator without excessive heat buildup (Joule heating).
- **Signal Isolation:** Sensitive analog lines (like the low-level signals for the Voltmeter and Ohmmeter) were routed away from the high-frequency PWM switching lines of the Function Generator. This prevents "crosstalk," where digital pulses would otherwise induce noise into your measurement readings.
- **Ground Plane:** A solid ground plane was utilized on the bottom layer. This provides a low-impedance return path for all signals, reducing electromagnetic interference (EMI) and ensuring the Raspberry Pi's logic "0" is consistent across the entire board.

5.2 Thermal Management

Several components in the E.T.B. generate significant heat, particularly the IRFZ34NS and IRF5803 MOSFETs used in the power supply.

- **Thermal Vias:** Large copper pours and thermal vias were placed around the MOSFET pads to act as a localized "heatsink," drawing heat away from the component and into the ground plane.
- **Component Spacing:** High-heat components were placed near the edges of the PCB to facilitate better airflow and prevent them from heating up the precision resistors in the R2R ladder, which could cause "thermal drift" in your DC reference.

5.3 Design for Manufacturability (DFM)

To ensure the board could be fabricated and soldered successfully, the following DFM rules were followed:

- **Clearance Rules:** A minimum clearance of 10 mil was maintained between all traces and pads to prevent solder bridges during the assembly phase.
- **Silkscreen Labeling:** Every header (J1, J2, etc.) and variable component (like the MCP4131) was clearly labeled on the silkscreen. This reduces the risk of incorrect component orientation during manual soldering.

- **Test Points:** Dedicated copper pads were placed at key nodes (e.g., +12V, -12V, and the R2R output) to allow for easy troubleshooting with a multimeter without needing to probe the delicate Raspberry Pi pins.

5.4 CAD Design Rules Check (DRC)

The final layout was validated using the KiCad Design Rules Checker (DRC).

- **Constraint Verification:** The DRC ensured that there were no "unconnected nets" or "clearance violations."
- **Netlist Integrity:** By syncing the schematic directly to the layout, the software guaranteed that every physical trace on the board matched the electrical logic of the design deliverables.

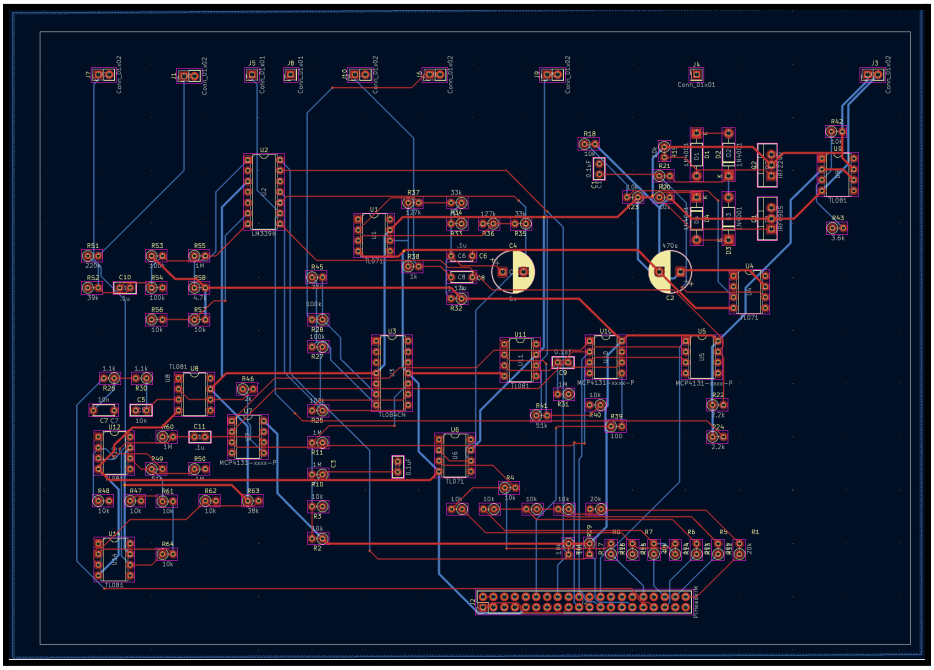


Figure 5.2 - Final PCB Layout

6. Ethics

All simulations, schematics, and software were developed by the team. Prewritten libraries are properly referenced in the Appendix.

The development of the Electronic Test Bench (E.T.B.) was governed by the ethical frameworks established by the National Society of Professional Engineers (NSPE). Throughout the design and implementation phases, our team remained committed to the primary directive of maintaining safety. This responsibility manifested most clearly in the development of the Raspberry Pi Protection Hat. By designing custom buffer circuits and voltage dividers, we ensured that any high-voltage inputs or accidental short circuits would be mitigated before reaching the Raspberry Pi. This ethical prioritization of hardware safety protects not only the equipment but also the students and educators who utilize the test bench in a laboratory setting.

In accordance with the principle of professional competence, our team structured the project tasks to align with our specific areas of engineering expertise. We maintained technical integrity by ensuring that all measurement data, specifically the readings from the Voltmeter and Ohmmeter, were processed with mathematical honesty. To avoid misleading the user, the software architecture incorporates precise nulling algorithms that account for the virtual ground offset. This ensures that the instrument provides an accurate representation of physical reality, fulfilling our ethical obligation to provide truthful and objective technical reports.

Finally, the project respects the intellectual property and contributions of the broader engineering community. All third-party Python libraries, such as “pigpio” and “spidev”, as well as the manufacturer datasheets for the integrated circuits used in the hardware, are properly disclosed within the documentation. By acknowledging these foundational resources in the Appendix and References sections, we uphold the standard of professional honor and transparency required of practicing engineers. This commitment to transparency extends to our self-critique, where we honestly evaluate the current limitations of our design to ensure future iterations can be improved upon with integrity.

Design decisions align with the principles outlined in the NSPE Code of Ethics.

7. Self-Critique

The team successfully engineered a functional multi-instrument test bench that met the core project requirements, though the development process revealed both significant technical strengths and areas for improvement. From a teamwork perspective, the group excelled in modular development by decoupling the hardware protection hat, the analog instrumentation circuits, and the Python-based user interface. This allowed for parallel troubleshooting, which proved essential given the complexity of integrating a Raspberry Pi 4 with high-voltage analog rails. The hardware design was robust, particularly the SAR ADC and the DC Reference scaling logic, which successfully managed to shift the DAC output to the required range using a difference amplifier configuration.

However, a significant technical challenge arose regarding the impact of component tolerances on the virtual ground. While the system achieved its functional goals, the slight resistance in the virtual ground rail introduced voltage offsets that necessitated software-side mathematical nulling. While this provided an effective fix, a more ideal hardware solution would have involved a more stable, active-buffered virtual ground to minimize the reliance on compensation algorithms. Furthermore, while the software architecture was highly intuitive and the hierarchical UI tree provided an excellent user experience, the system's sampling rate was occasionally limited by Python’s processing overhead. The logic for the Successive Approximation Register was sound, but the execution speed could have been further optimized through lower-level programming or C-extensions to provide a more real-time response for the oscilloscope and voltmeter functions.

7.1 Revision Plan

If granted a second iteration, the team would focus on hardware refinement and system efficiency to transition the E.T.B. from a functional prototype to a professional-grade instrument. One of the primary design changes would be the implementation of an active virtual ground using a high-current op-amp buffer. This would ensure a rock-solid reference that does not fluctuate under the varying loads of the R-2R ladder, thereby increasing overall measurement precision. Additionally, the team would move toward hardware-level signal conditioning. By incorporating active Sallen-Key low-pass filters on the input stages of the Voltmeter and Ohmmeter, we could eliminate high-frequency noise before it reaches the ADC, reducing the need for the processing-heavy software averaging used in the current design.

The transition from a class project to a real-world product would require a shift toward commercial safety and manufacturing standards. A commercial version of the E.T.B. would include dedicated ESD protection diodes on all user-facing terminals and galvanic isolation to meet formal safety certifications. The user interface would likely be moved to a custom Linux kernel to provide a "turn-key" appliance feel, hiding the OS background from the end user. Finally, the manufacturing process would move away from through-hole components in favor of Surface Mount Technology. This change would not only reduce the physical form factor of the device but also decrease assembly costs and improve signal integrity by minimizing trace lengths and parasitic capacitance in the analog front-end.

7.2. Individual Reflections

Justin - Overall, I think the project was a very fun experience to use the Raspberry Pi. The idea of combining hardware and software to implement something was very interesting for me. Although it took a lot of time and effort, after completing Checkpoint A-, it was worth all the effort we put in during the semester. At first, I didn't even know where to start with the project, but as we completed each deliverable, things seemed to make a lot more sense and I understood how each component worked with each other along with the software. It was also a good opportunity for me to learn to code in Python as I have only used Java before this project, which will definitely help me in any future projects I want to work on.

Avery - Working on this project has been very educational. I feel like I learned more from this class about engineering than any previous. At the beginning of the semester everything felt very daunting and now at the end it all feels approachable. I also think that I will be less likely to damage future projects because I made a lot of easily corrected mistakes with this one. I burned out more components than anyone in the group, but I think I learned from it, so it's okay. Learning how to make schematics and PCB files has been great as well. I feel much more prepared for future projects, just for the fact that I'll know how to quickly represent them visually. All the technical elements of this project have been awesome to learn.

Jakob - This project has been a fun experience for me. It has let me see some of the flaws and strengths I have when working within an engineering environment. I feel like this has shown me and guided me into what I want to do for my future career. When you break down the project into digestible parts it becomes easier to navigate and know where to start. I think that is the biggest takeaway I had. It also requires a lot of time management, and some people will always know more than you.

Allan - This was a cool project. I enjoyed working on it. Seeing all the theory from past labs finally come together into one functioning system was exciting and made everything feel more real. It pushed me to slow down, think through problems, and understand how each part of the hardware and software interacted. I also learned a lot about working in a team. Everyone brought different perspectives and strengths, and that helped me improve how I communicate and contribute. Even when things did not go smoothly, it felt good knowing we were building something meaningful together.

Sam - This was one of the more enjoyable classes that I have taken through my college career. The freedoms that we were given to create our project allowed us to learn through hands-on trial and error and gain a deeper understanding of what it actually means to apply the knowledge that we've built up to this point. While challenging at times, seeing your hard work and critical thinking come together in a functional system is an incredibly rewarding experience. Working as a group has also given me a better understanding of how to navigate the strengths of my team and adapt to work in the most efficient way possible. This project also forced me a little out of my comfort zone working with coding languages that I am not as familiar with and creating designs that I had never previously implemented. I am grateful for all that I can take away from this class and proud of myself and my team for conquering such a demanding task.

8. Appendix

A.1 – GitHub Repository and Revision History

https://github.com/Herring-UGAECSE-2920-S26/Group-4/commits

A.2 - Component Datasheets

https://www.ti.com/lit/ds/symlink/lm339-n.pdf (LM339N)

https://www.microchip.com/en-us/product/mcp4231 (MCP4231)

https://ww1.microchip.com/downloads/en/DeviceDoc/22060b.pdf (MCP4131)

A.3 - Hardware Implementation

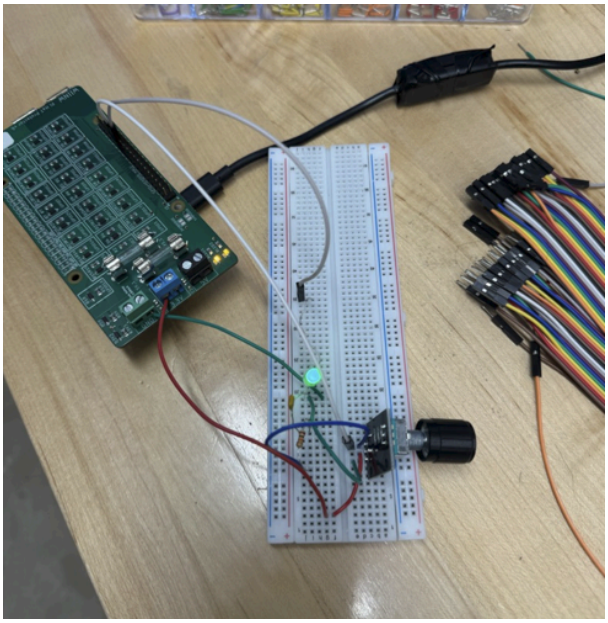


Fig. 1 - Rotary Encoder Hardware

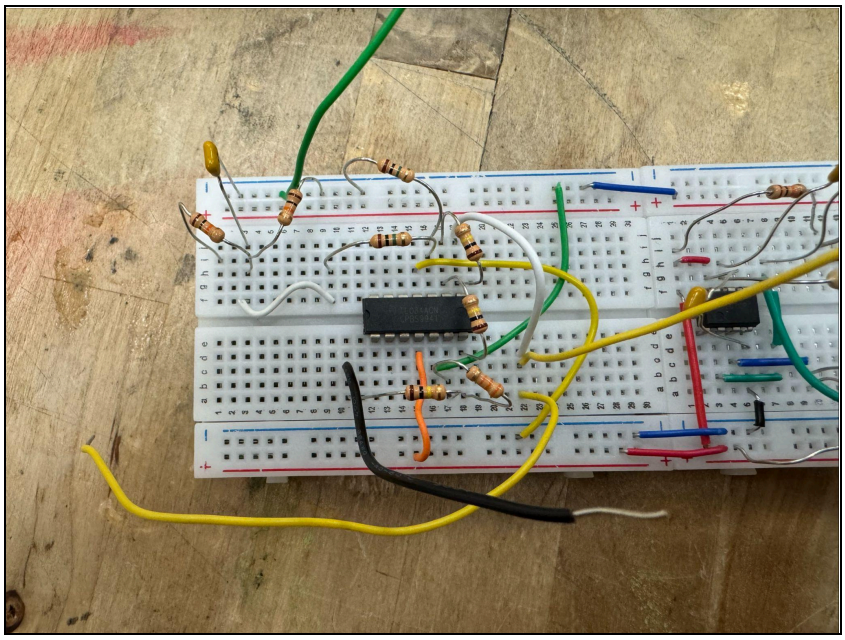


Fig 2 - Voltmeter Hardware

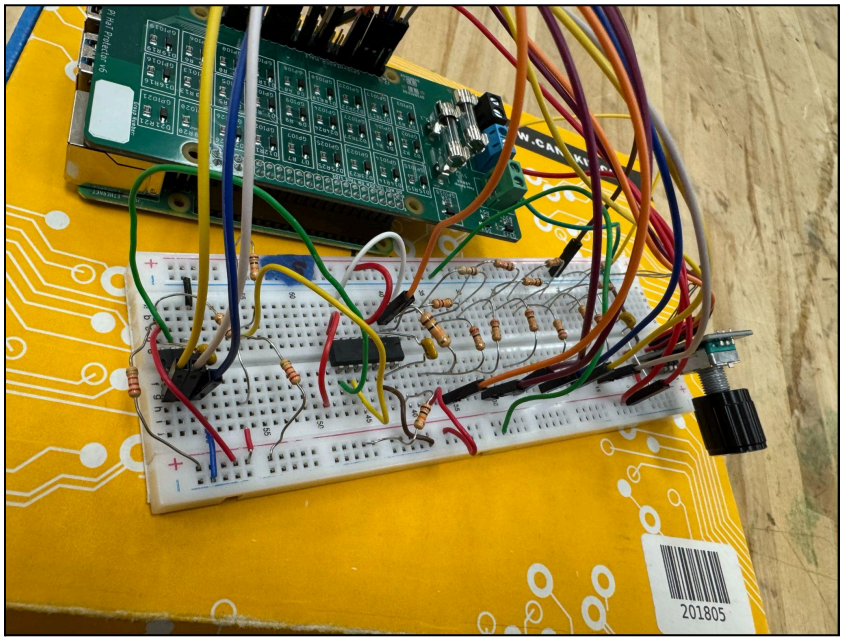


Figure 3 - SAR 6-Bit ADC Hardware

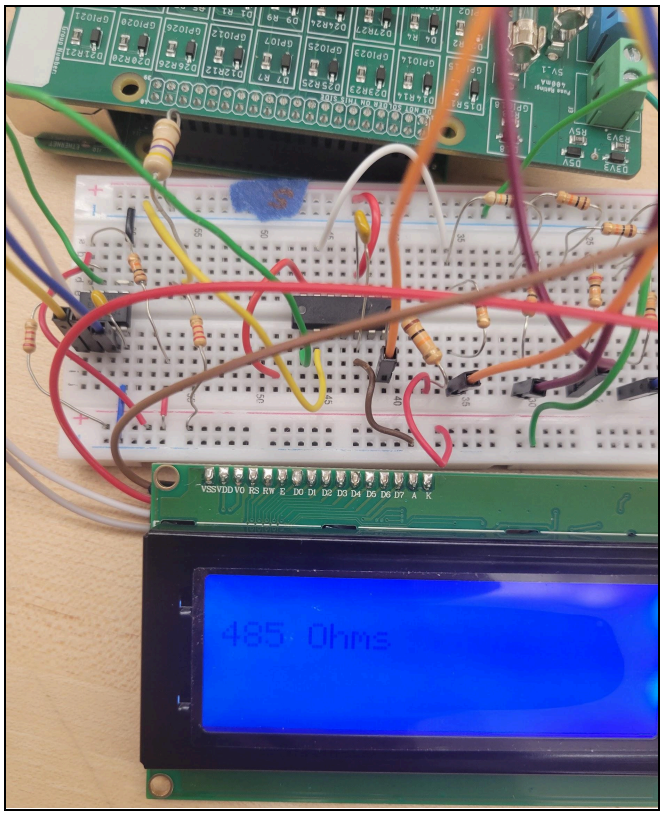


Figure 4 - Ohmmeter Hardware